Product Requirements Document

Auth & Account Security Hardening — Storey ConTech ERP

Author: Claude (with Karun) **Date:** 2026-05-15 **Status:** Server-side shipped; frontend in PR **Scope:** Authentication, account integrity, row-level security, signup flow, deployment resilience

1. Background & Problem Statement

Storey is a multi-tenant ConTech SaaS used by contractors and field crews in Northeast India. As we approach Play Store submission, an internal audit of the Supabase login surface surfaced several **exploitable security holes** alongside a set of UX and reliability gaps that would visibly degrade the launch experience.

Most issues were a single browser-console paste away from full platform compromise. They had to ship before any production user growth.

1.1 Discovered issues (severity-ranked)

#	Issue	Severity	Status
A	Direct `UPDATE profiles SET role='superadmin'` from any signed-in user	🔴 P0	Shipped
B	SMS OTP account takeover (overwrite victim's phone, log in as them)	🔴 P0	Shipped
C	Unlimited OTP attempts (brute-force in minutes)	🔴 P0	Shipped
D	Magic-link URL leaked in JSON response body	🟠 P1	Shipped
E	User enumeration via "User not found" 400	🟠 P1	Shipped
F	No server-side OTP rate limit (Twilio cost / DoS)	🟠 P1	Shipped
G	Orphaned auth users when tenant insert fails post-signup	🟡 P2	Shipped
H	Intermittent 404s after redeploy (stale JS chunks)	🟡 P2	Branch only
I	Multi-tab signOut sync broken	🟡 P2	Branch only
J	Profile-fetch errors swallowed (no user feedback)	🟢 P3	Branch only
K	OAuth `?error=` silently redirected to login	🟢 P3	Branch only

2. Goals

1. **Security:** close every exploit path discoverable from a browser console without insider access.
2. **Integrity:** eliminate orphan/inconsistent records introduced by partial flows.
3. **Reliability:** users should never see a blank page or unexplained redirect during normal use.
4. **Privacy:** no API response should leak which emails are registered or which phone belongs to whom.
5. **Backwards compatibility:** existing users must keep working — no forced re-onboarding.

Non-goals (this release)

- Replacing Supabase for auth — too disruptive at this stage.
- Adding MFA beyond SMS OTP.
- A self-service "change my phone" admin flow (deferred — manual support escalation for now).
- Email verification enforcement for new signups (out of scope; left to Supabase project config).

---

3. Functional Requirements

3.1 Authentication

ID	Requirement	Acceptance
AUTH-1	Email/password sign-in (existing flow)	Unchanged
AUTH-2	Google OAuth sign-in (existing flow)	Unchanged; provider errors surfaced
AUTH-3	SMS OTP sign-in	Requires `profiles.phone` to be set AND match submitted number
AUTH-4	Send-OTP rate limit	Server rejects re-send within 60 s for the same user
AUTH-5	Verify-OTP attempt cap	Hard 5-attempts-per-record; further attempts return 429
AUTH-6	Generic error responses	Unknown emails / mismatched phones return identical 200 success
AUTH-7	Session token issuance	`token_hash` only — never the full magic-link URL

3.2 Account & Tenant Lifecycle

ID	Requirement	Acceptance
ACCT-1	New contractor registration	Atomic — auth user created and tenant row created in one server call; auth user rolled back on tenant failure
ACCT-2	Google-OAuth contractor first-time tenant creation	`link_owner_to_tenant` trigger continues to fire and assign `tenant_id` + `role='contractor'`
ACCT-3	Phone enrollment	Auth-gated edge function; sets `profiles.phone` exactly once
ACCT-4	Phone change	Locked to superadmin / support escalation in this release
ACCT-5	Multi-tab sign-out	Signing out in one tab propagates to all open tabs within one route change

3.3 Row-Level Security

ID	Requirement	Acceptance
RLS-1	`profiles.role` immutability	Non-superadmin / non-service-role caller cannot change it; trigger raises exception
RLS-2	`profiles.tenant_id` immutability	Same constraint as RLS-1
RLS-3	`profiles.phone` / `phone_verified` / `auth_method` immutability	Same constraint; only enrollment edge function can change them
RLS-4	`profiles.full_name` user-editable	Settings page continues to write this freely
RLS-5	Trigger-fired updates from trusted SECURITY DEFINER functions	Allowed via transaction-local bypass flag

3.4 Reliability

ID	Requirement	Acceptance
REL-1	Stale chunk after redeploy	Tab from previous build auto-reloads once instead of blanking
REL-2	404 page	Shows the requested path + contextual CTA based on session
REL-3	Profile fetch failure	Visible amber banner with Retry button instead of empty profile
REL-4	OAuth provider error	Friendly message instead of silent `/login` redirect
REL-5	StrictMode dev double-init	`authStore.init` returns the same promise to both invocations

---

4. Architectural Decisions

4.1 Trigger-based RLS enforcement instead of column-level policies

Chose: A `BEFORE UPDATE` trigger that raises exceptions on protected columns. **Alternative:** Per-column `FOR UPDATE OF (col)` RLS policies. **Why:** One readable, well-commented function vs. ~5 entangled policies. Bypass path for legitimate trusted callers (`link_owner_to_tenant`) is also clearer.

4.2 Transaction-local bypass for legitimate trigger-fired updates

Chose: `set_config('app.bypass_profile_immutability', 'on', true)` inside `link_owner_to_tenant` , read by the immutability trigger. **Alternative:** Whitelist by `pg_trigger_depth()` or function-name inspection. **Why:** `is_local=true` guarantees the flag auto-clears at end-of-transaction, so it can't leak. Trusted callers explicitly opt in by setting the flag.

4.3 Separate enrollment endpoint instead of allowing implicit OTP enrollment

Chose: New `enroll-phone-otp` and `verify-phone-enrollment` edge functions, auth-gated. **Alternative:** Continue allowing implicit phone enrollment on first OTP login. **Why:** Implicit enrollment + email-known attacker = trivial account hijack. Splitting enrollment behind an authenticated session means an attacker would need to already be the account owner.

Trade-off: Existing users with `profiles.phone = NULL` get a silent "no SMS received" if they try OTP login without first enrolling. Acceptable because (a) they have email/password and Google as alternatives, (b) Settings has a clear enrollment UI.

4.4 Server-side atomic registration via edge function

Chose: `register-tenant` edge function that creates auth user + tenant in one call with rollback. **Alternative:** Keep client-side two-step. **Why:** Postgres transactions don't span the auth schema (`auth.users` is managed by Supabase's auth service). The edge function compensates by manually deleting the auth user if the tenant insert fails.

4.5 Stale-chunk recovery via `vite:preloadError` instead of cache-busting

Chose: Scoped `vercel.json` rewrite + sessionStorage-gated auto-reload. **Alternative:** Service worker, cache headers, or chunk-hashing changes. **Why:** Single-listener, ~10 lines of JS, ~1 line of Vercel config. Service-worker route was significantly more complex for a problem that only manifests transiently after deploys.

---

5. Implementation Status

5.1 Live on production now

```
DB migrations applied to zgvbogxibiilnblmuohg:
  20260515000000  profile field immutability trigger
  20260515000001  link_owner_to_tenant bypass bridge
  006_budget_lines (history reconciled — was applied manually)

Edge functions deployed:
  send-sms-otp           v14   (updated)
  verify-sms-otp         v22   (updated)
  enroll-phone-otp       v2    (new)
  verify-phone-enrollment v2   (new)
  register-tenant        v2    (new)
```

5.2 In branch `claude/heuristic-kepler-947fd0` (awaiting merge to `main`)

```
bdba54f  fix(security): bridge link_owner_to_tenant past the immutability trigger
4b16228  fix(security): lock sensitive profile fields against direct UPDATE
914f906  fix(app): stale-chunk 404 recovery and friendlier NotFound
0ba5048  feat(auth): phone enrollment flow, atomic tenant signup, store hardening
19cb2fe  fix(auth): close SMS OTP takeover + brute-force vectors
```

Branch is **5 ahead, 43 behind** `main`. Non-trivial overlap with newer native-Capacitor work on `main` — merge will require deliberate conflict resolution.

5.3 Planned / outstanding

Item	Priority	Owner	Notes
Play Store icon + screenshots upload	P0 launch-blocker	Karun (manual)	Console automation blocked by Google CDP restrictions
Resolve frontend branch divergence from `main`	P1	Karun / Claude	Either rebase + manual resolve OR cherry-pick security-only commits
Add `phone_verifications_user_created_idx` index	P2	Backend	Prevents rate-limit query degradation at scale
"No phone on file — enroll in Settings" inline hint in SMSOTPLogin	P2	Frontend	UX clarity for users who can't OTP-login yet
Audit other tables' RLS for the same `WITH CHECK` gap	P2	Security	We checked `profiles`; `tenants`, `sites`, `pending_invites`, `budget_lines` deserve the same scrutiny
Dedicated "Change my phone" admin flow	P3	Support tools	Currently support-only via direct DB
Migration cleanup — formalise the manually-applied `006_budget_lines` history	P3	Backend	Already reconciled, just cosmetic

---

6. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Existing users without phone get stuck on SMS OTP login	Medium	Low	Email/password + Google alternatives; Settings enrollment is visible. Follow-up: inline hint.
Frontend merge introduces conflicts	High	Medium	Server side is already live and verified; frontend is polish. Can be cherry-picked or re-applied.
`link_owner_to_tenant` future modifications drop the bypass flag	Low	Medium (signup breaks)	Trigger function has inline comment explaining the bypass. End-to-end test in this PR documents the contract.
Trigger blocks a legitimate codepath we haven't audited	Low	Medium	All client-side UPDATE sites grepped — only `Settings.jsx` (full_name) touches profiles. Anything else would surface immediately as an exception.
`register-tenant`'s `email_confirm: true` bypasses Supabase's email-verify setting	Low	Low	Only called by our new signup flow; current main signup doesn't use it. Revisit if email-verify ever becomes a requirement.

---

7. Success Metrics

Metric	Baseline	Target
Successful privilege-escalation exploit attempts	Unknown — exploitable	0 (immutability trigger active)
Orphaned `auth.users` without matching tenant	Unmeasured	0 (atomic register)
OTP brute-force attempts beyond cap	Unmeasured	Hard-capped at 5
User-reported "blank page after refresh"	Anecdotal post-deploy	0 — auto-reload recovers
Multi-tab signOut "still logged in" complaints	Anecdotal	0
SMS OTP login attempts hitting the silent "no phone" path	Unmeasured	Track and tune the UX hint

Operational telemetry to add (P2):

- Counter on each edge function for 200 / 400 / 401 / 429 / 500 responses.
- Dashboard for `phone_verifications.attempts` distribution to spot brute-force patterns.

---

8. Open Questions

1. **Should email verification be enforced before sign-in?** Currently `register-tenant` sets `email_confirm: true`, bypassing verification. If we want verification, this needs revisiting.
2. **What's the right UX for users who try SMS OTP without phone enrolled?** Generic "If details are correct, OTP sent" preserves privacy but is confusing. Option: show a one-time hint after 2 failed attempts pointing to Settings.
3. **Do we need a Phone-change flow at launch, or is support-only OK?** Currently locked to superadmin / support escalation.
4. **Same RLS audit on `tenants`, `sites`, `pending_invites`, `budget_lines`?** This audit only covered `profiles` — sibling tables may have similar gaps.
5. **Migration tracking hygiene** — `006_budget_lines` was manually applied. Worth retroactively documenting what other migrations may have been ad-hoc'd, to avoid future "out of order" headaches.

---

9. Glossary

- **PostgREST**: Supabase's REST API layer over Postgres. Most client-side `supabase.from(...)` calls go through this.
- **RLS**: Row-Level Security. Postgres policies that filter visible rows by the calling user's JWT.
- `auth.role()` / `auth.uid()`: Supabase helpers that read the current JWT's role and user-id claims.
- **Service role**: The privileged Supabase API key used by edge functions to bypass RLS. Never exposed client-side.
- `SECURITY DEFINER`: A Postgres function that runs with its owner's privileges (typically `postgres`), regardless of who called it.

Appendix A — Test Plan (paste into PR description)

- [ ] **Privilege escalation closed** — sign in as any non-admin, paste into browser console: `await supabase.from("profiles").update({ role: "superadmin" }).eq("id", (await supabase.auth.getUser()).data.user.id)` → expect a Postgres error, not `data: [...]`.
- [ ] **Phone enrollment works** — sign in fresh, Settings → "Add phone number", complete OTP → `profiles.phone` is set, card flips to "Verified phone" state.
- [ ] **Phone enrollment is locked once set** — try a second enrollment with a different number → 409 "Phone already enrolled".
- [ ] **SMS OTP login requires enrolled phone** — for a user with `profiles.phone = null`, attempt OTP login → generic success returned but no SMS, verify never accepts.
- [ ] **Attempt cap fires** — request OTP, enter wrong code 5 times → 429 on the 5th.
- [ ] **Registration is atomic** — force a tenant-name failure → no orphan in `auth.users`.
- [ ] **OAuth cancel UX** — visit `/auth/callback?error=access_denied&error_description=cancelled` → friendly message, not silent redirect.
- [ ] **Multi-tab signOut sync** — sign out in tab A → tab B reflects after next route change.
- [ ] **Stale-chunk recovery** — after deploy, old tabs auto-reload once instead of blank page.
- [ ] **Google-OAuth contractor signup** — sign up via Google as a new contractor → tenant gets created, profile gets linked, no "tenant_id immutable" error.

Appendix B — Deploy commands reference

```
# Database
supabase db push                              # applies pending migrations
supabase migration repair --status applied N  # mark a manually-run migration as applied

# Edge functions
supabase functions deploy <name1> <name2> ... # deploy one or more functions

# Smoke checks
supabase db query --linked -o table "<sql>"   # ad-hoc SQL against the linked project

# Frontend
git push origin <branch>                      # Vercel auto-deploys on push to main
```